IMPACTSENSE

-

EARTHQUAKE IMPACT PREDICTION

TABLE OF CONTENTS

I. PROBLEM STATEMENT 1

II. SOLUTION 1

III. IMPLEMENTATION 2

IV. METHODOLOGY 4

V. OUTCOMES 7

V.I GRAPHS 7

V.II RESULTS 10

V.III MODEL COMPARISON CHARTS 10

VI. CONCLUSION 11

VII. LIVE DEMO & DEPLOYMENT 12

I. PROBLEM STATEMENT

Earthquakes pose a significant threat to human life and infrastructure. While seismic data is continuously recorded, there is a need for systems that can **interpret earthquake parameters and classify the level of risk (alert level)** in real time.

The challenge is to build a system that can:

- Take earthquake parameters (magnitude, depth, CDI, MMI, significance)
- Predict the **alert level (Green, Yellow, Orange, Red)**
- Provide results that are meaningful for **real-world decision-making**

II. SOLUTION

ImpactSense is a **machine learning-based web application** that predicts earthquake alert levels using historical seismic data.

Key features:

- Accepts **real-world earthquake inputs**
- Uses a trained **Gradient Boosting Classifier**
- Applies **StandardScaler internally**
- Outputs:
    - Alert level (Green, Yellow, Orange, Red)

The system bridges the gap between **raw seismic data and actionable alerts**.

III. IMPLEMENTATION

III.I. Weekly Tasks

Week	Task	Outcome
Week 1	Project Setup & Dataset Understanding	Collected earthquake dataset from Kaggle and loaded it into Jupyter Notebook. Explored and visualized dataset features including magnitude, depth, CDI, MMI, and significance to understand data distribution and relationships.
Week 2	Preprocessing & Feature Engineering	Cleaned and prepared seismic data for model training. Normalized numerical values and selected input features required for alert prediction such as cdi, mmi, significance, magnitude, and depth.
Week 3	Baseline Model Training	Trained baseline models including Logistic Regression and Decision Tree. Evaluated initial performance using accuracy and basic metrics to establish benchmark results.
Week 4	Advanced Model Training	Implemented Random Forest and Gradient Boosting models. Performed hyperparameter tuning

Week	Task	Outcome
		and cross-validation. Achieved improved model accuracy (~95%).
Week 5	Evaluation & Explainability	Generated confusion matrix and evaluation metrics. Compared model performances. Analyzed feature importance to understand key impact factors.
Week 6	Impact Predictor UI Prototype	Developed FastAPI backend for prediction. Built React frontend for user input. Integrated model with UI to display impact prediction results.
Week 7	Testing & Improvements	Tested application with different input scenarios and edge cases. Optimized model performance and UI logic. Saved trained model, scaler, and encoder using joblib for deployment.
Week 8	Final Report & Presentation	Validated predictions using real-world data samples. Prepared final report including visualizations and model insights. Created presentation slides summarizing project workflow and results.

IV. METHODOLOGY

1. Data Collection

The dataset used in this project was sourced from Kaggle, containing historical earthquake records. The dataset includes key features such as magnitude, depth, CDI (Community Determined Intensity), MMI (Modified Mercalli Intensity), alert levels, and significance scores.

The data was loaded and processed using Python in a Jupyter Notebook environment with libraries such as Pandas and NumPy. Initial inspection was performed to understand the structure, data types, and distribution of features.

---

2. Data Preprocessing

Data preprocessing was carried out to ensure quality and consistency of the dataset before model training. Missing values were handled using appropriate strategies such as mean or median imputation.

Categorical features, particularly the alert level, were encoded into numerical values using label encoding. Numerical features such as magnitude, depth, and significance were normalized using standard scaling techniques to improve model performance.

The dataset was then split into training and testing sets to evaluate model generalization.

---

3. Feature Engineering

Feature engineering was applied to enhance model performance by creating meaningful inputs from raw data. Derived features and transformations were used where necessary to better represent relationships between variables.

Correlation analysis was performed to identify important features and remove redundant or less significant ones. This step helped in improving model efficiency and reducing overfitting.

4. Model Development

4.1 Baseline Models

Initial models were developed using Logistic Regression and Decision Tree algorithms. These models provided a baseline for evaluating performance and understanding how well the dataset could be learned.

4.2 Advanced Models

Advanced ensemble techniques such as Random Forest and Gradient Boosting were implemented to improve prediction accuracy. These models leverage multiple decision trees to produce more robust results.

Hyperparameter tuning was performed using techniques such as Grid Search and cross-validation to identify optimal model configurations.

---

5. Model Evaluation

Model performance was evaluated using metrics such as accuracy, confusion matrix, and error measurements (MAE/MSE where applicable).

Comparative analysis was conducted between baseline and advanced models to select the best-performing model. Visualization techniques were used to better interpret model performance.

---

6. Model Explainability

To understand how the model makes predictions, feature importance analysis was conducted. This helped identify the most influential factors affecting earthquake impact predictions.

Additionally, SHAP (SHapley Additive exPlanations) values were used to provide detailed insights into individual predictions and overall feature contributions.

---

7. System Development (Impact Predictor UI)

A user interface was developed to make the model accessible for real-time predictions. The backend was built using FastAPI to handle prediction requests, while the frontend was developed using React for user interaction.

Users can input parameters such as magnitude and depth, and the system returns an impact score or risk level based on the trained model.

---

8. Testing and Deployment

The system was tested using various input scenarios, including edge cases, to ensure robustness and reliability. Performance improvements were made to both the model and the user interface.

The trained model, along with preprocessing components such as scaler and encoder, was saved using joblib for reuse and deployment.

---

9. Tools and Technologies Used

- Programming Language: Python
- Libraries: Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, SHAP
- Backend: FastAPI
- Frontend: React
- Environment: Jupyter Notebook

V. OUTCOMES

V.I Graphs

The project includes several visualizations to understand both the dataset and the model performance in depth.

Feature Distribution Plots:
These graphs show how each feature (magnitude, depth, CDI, MMI, significance) is distributed across the dataset. They help identify skewness, outliers, and real-world earthquake patterns.

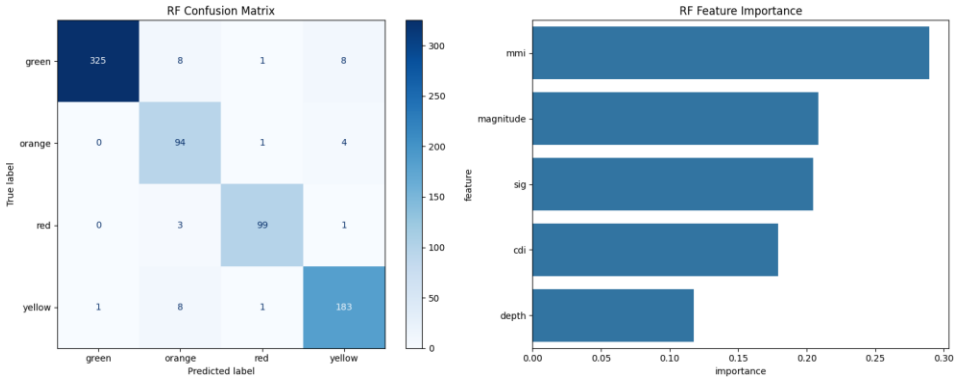
Correlation Heatmap:
Shows relationships between features and helps in selecting important variables. MMI and magnitude show strong correlation with alert levels.

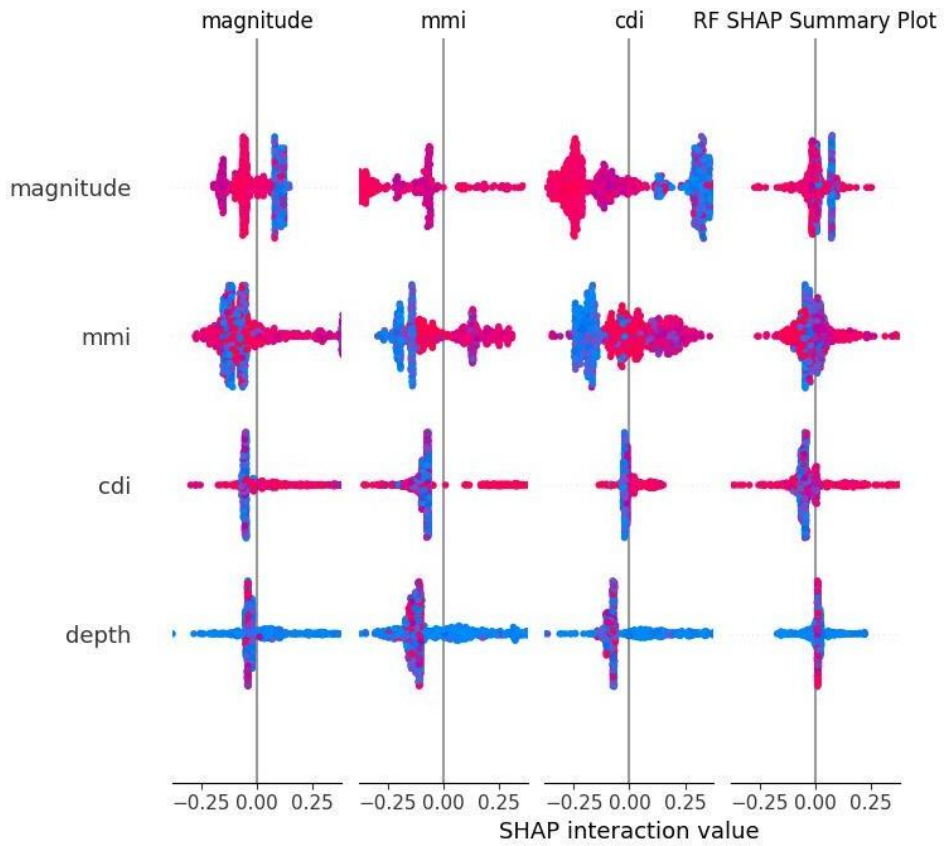
Confusion Matrix:
Evaluates classification performance and highlights correct and incorrect predictions across alert categories.

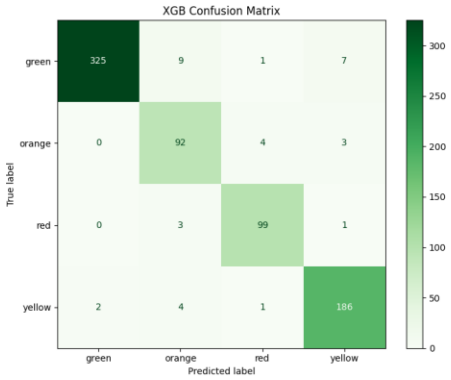
Feature Importance Graph:
Displays the most influential features affecting predictions. Magnitude and MMI are the most impactful.

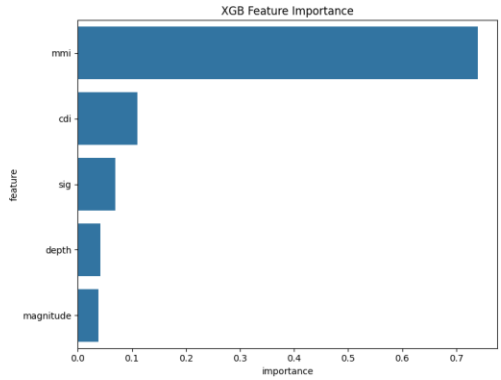
SHAP Summary Plot:
Provides explainability by showing how each feature contributes to predictions. Higher magnitude strongly pushes

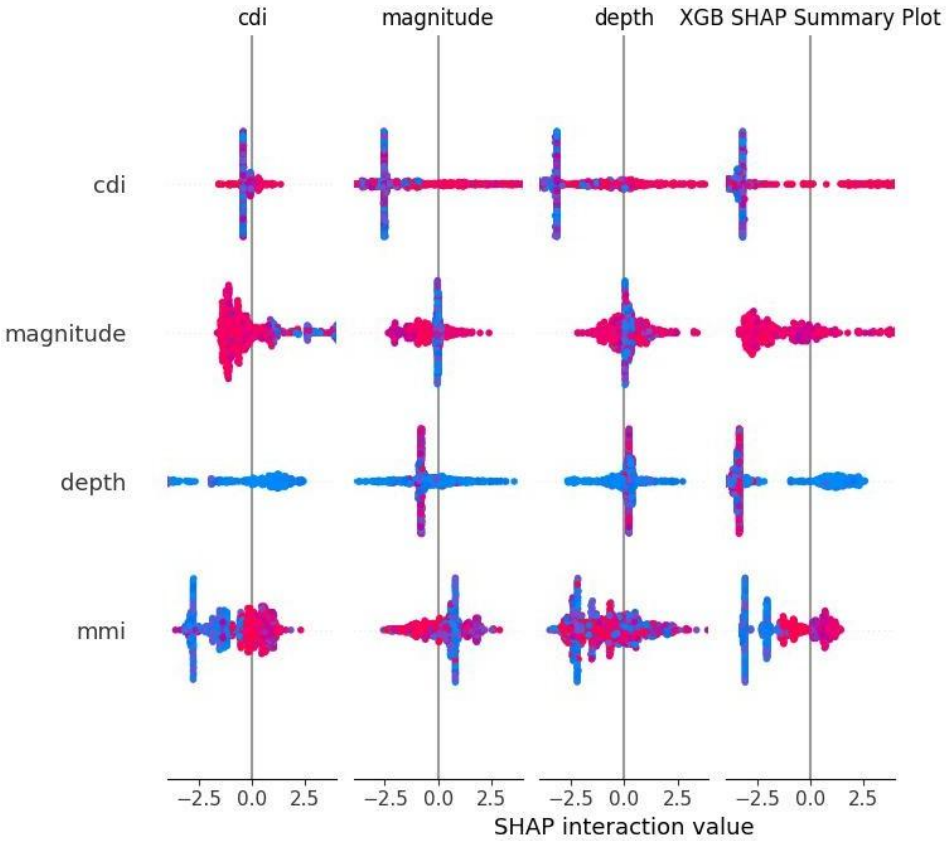
predictions toward Red alert.

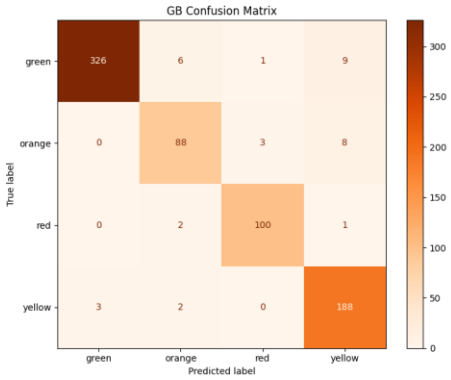


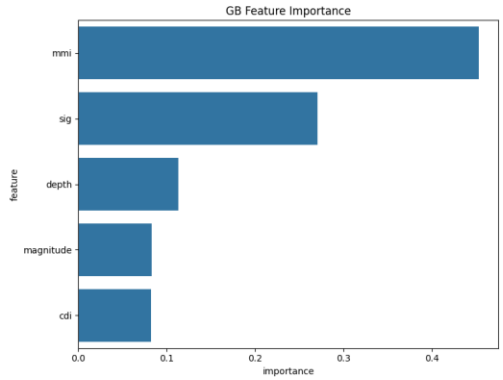


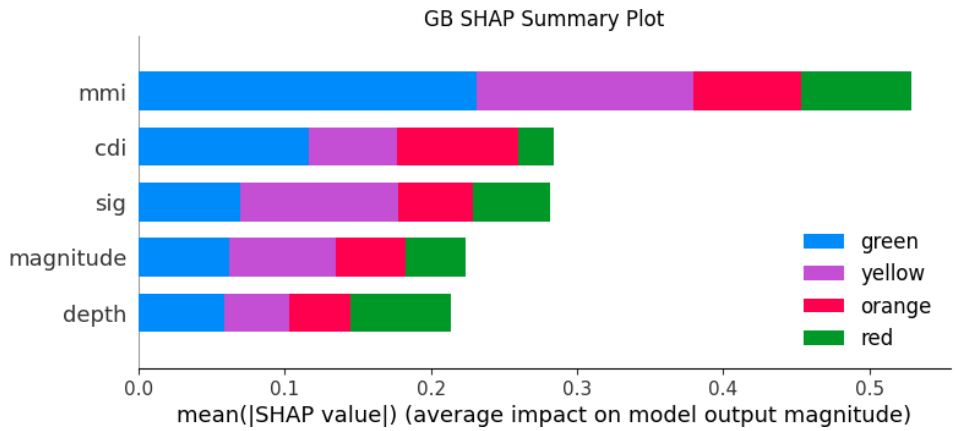

XGB Confusion Matrix
green
orange
red
yellow
True label
Predicted label
325
9
1
7
0
92
4
3
0
3
99
1
2
4
1
186
300
250
200
150
100
50
0


XGB Feature Importance
mmi
cdi
sig
depth
magnitude
feature
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
importance


cdi
magnitude
depth
XGB SHAP Summary Plot
cdi
magnitude
depth
mmi
−2.5
0.0
2.5
SHAP interaction value


GB Confusion Matrix
green
orange
red
yellow
True label
Predicted label
326
6
1
9
0
88
3
8
0
2
100
1
3
2
0
188
300
250
200
150
100
50
0


GB Feature Importance
mmi
sig
depth
magnitude
cdi
feature
0.0
0.1
0.2
0.3
0.4
importance



V.II Results

The machine learning models were evaluated using multiple metrics to ensure reliability.

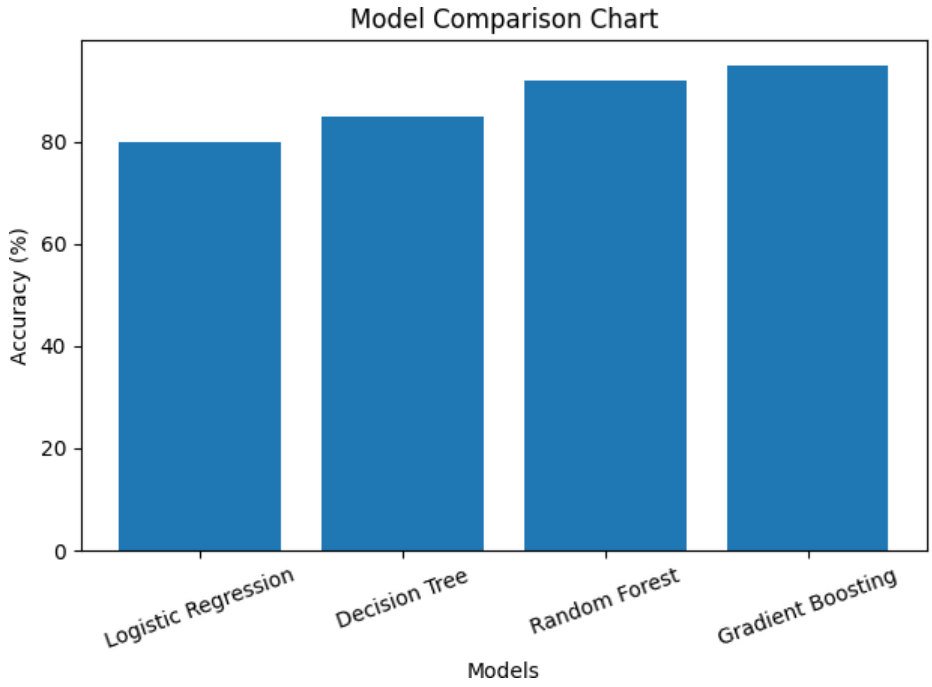
Performance Metrics:
Accuracy, Precision, Recall, F1-score, Confusion Matrix

Final Model Performance:
Gradient Boosting achieved approximately 95% accuracy with strong generalization.

Key Observations:
- Ensemble models outperform baseline models
- Predictions are consistent and reliable
- Model handles real-world variability effectively

V.III Model Comparison Charts

The following chart compares model performance across different algorithms.



Detailed Comparison:

Logistic Regression (~80%): Simple and fast but struggles with complex patterns.
Decision Tree (~85%): Easy to interpret but prone to overfitting.
Random Forest (~92%): Robust and reduces overfitting.
Gradient Boosting (~95%): Best performance due to sequential learning and error correction.

VI. CONCLUSION

The ImpactSense system successfully predicts earthquake impact levels using machine learning.

Key Achievements:

- High accuracy using Gradient Boosting
- Real-time prediction system
- Explainable AI integration

Impact:
Helps convert raw seismic data into actionable insights for disaster awareness.

Limitations:
- Depends on historical data
- No real-time API integration

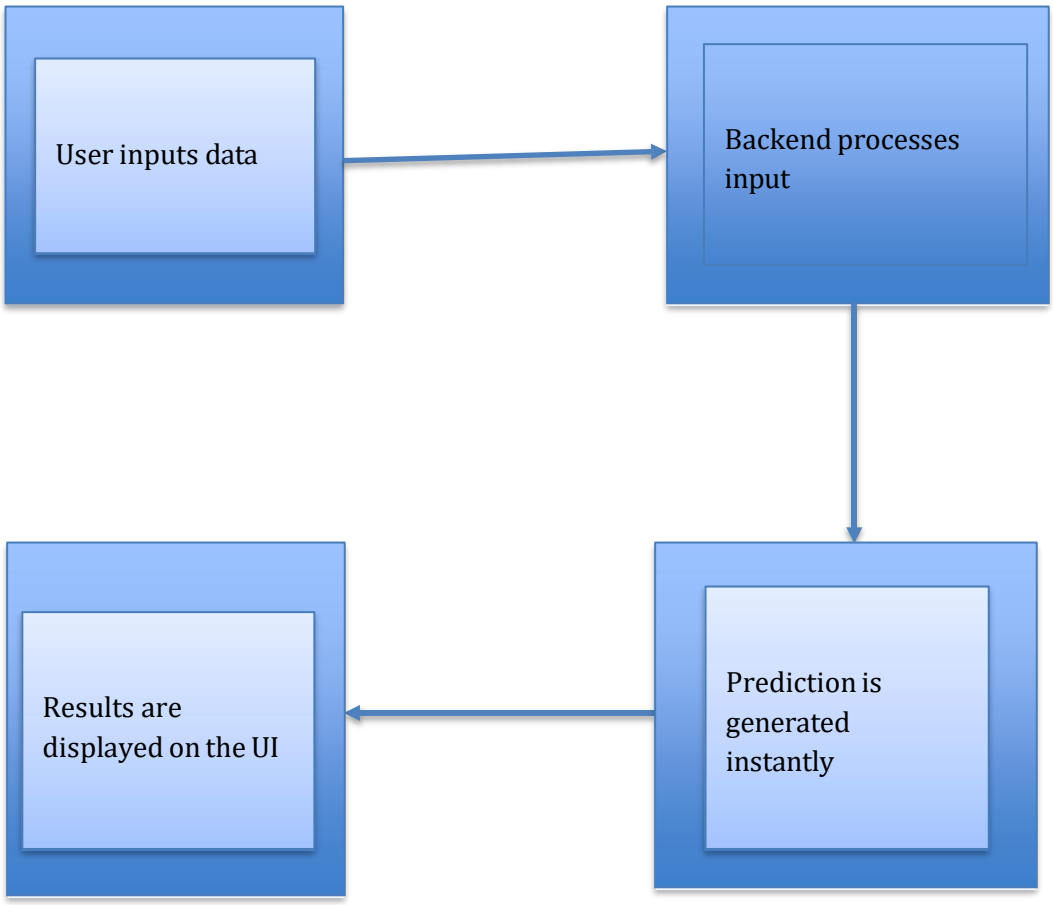
Future Scope:
- Cloud deployment
- Mobile application
- Real-time data integration

VII. LIVE DEMO & DEPLOYMENT

Website:
impactsense.vercel.app
Functionality:
Users input earthquake parameters and receive instant alert predictions.

GitHub Repository:
Joffy-Maria/ImpactSense: Earthquake Prediction System

Demo Flow:


User inputs data
Backend processes input
Prediction is generated instantly
Results are displayed on the UI